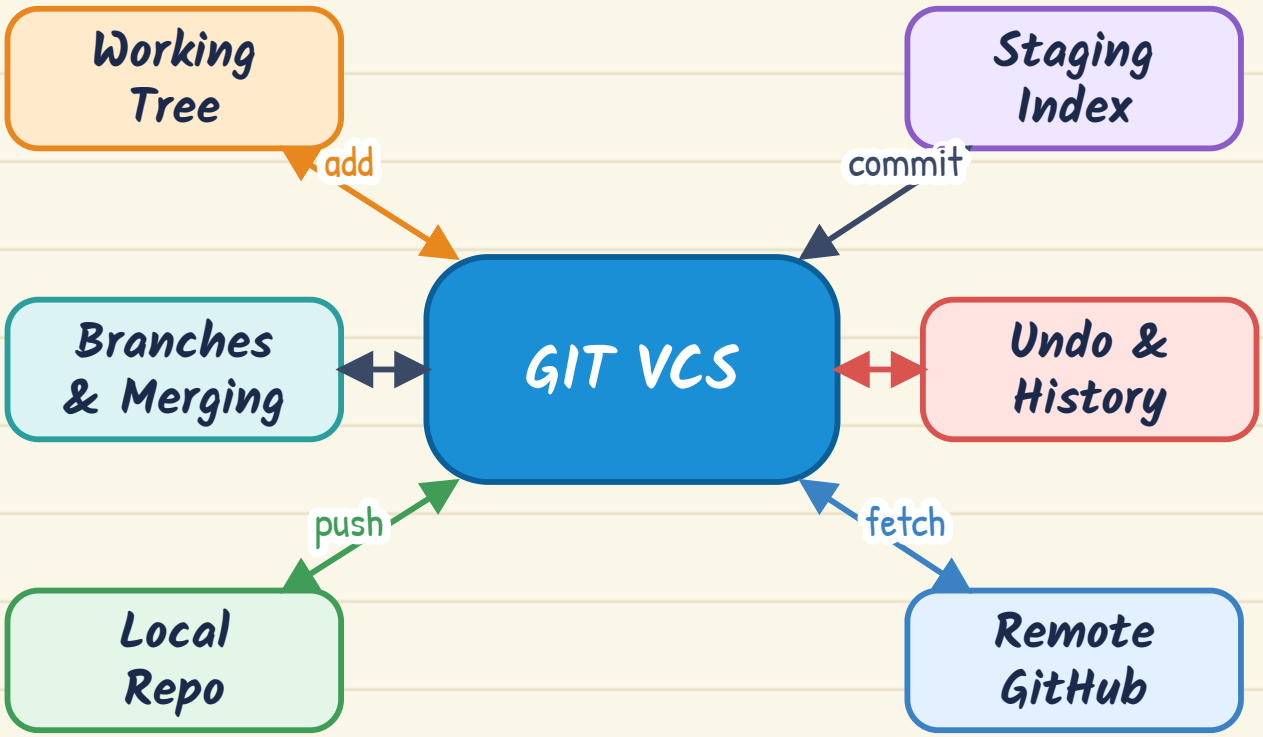
GIT

QUICK NOTES

(Version Control Basics)



BEGINNER TO INTERMEDIATE

SWIPE TO NEXT →

Roadmap

GIT
VCS Basics

4-page rapid mastery of Git core mental models, branching, and collaboration

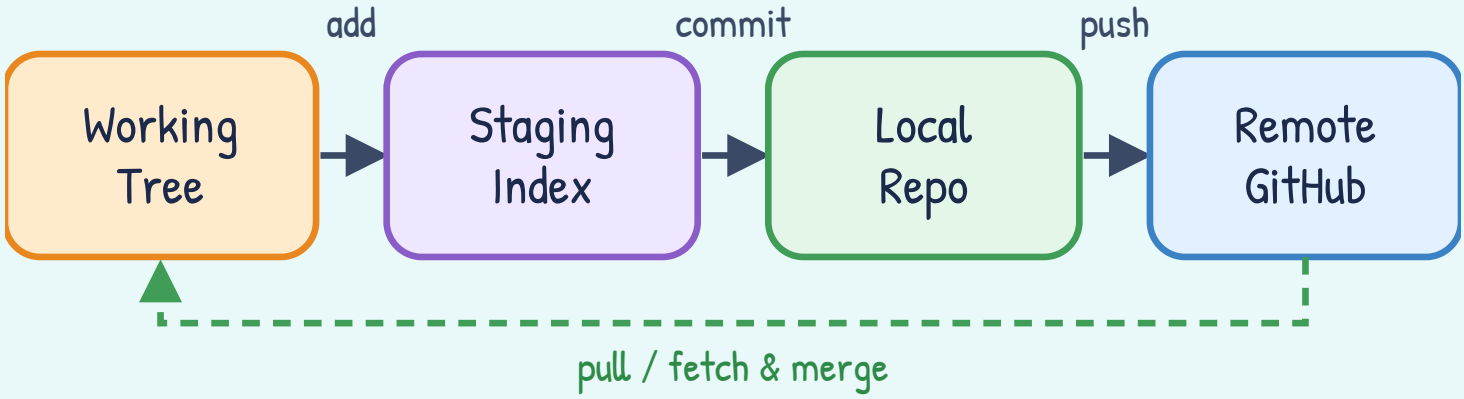
3 The Three Trees: Work, Stage & Commit
Understanding how Git moves changes through working directory, index, and repository

4 Branching, Merging & Moving HEAD
How Git tracks divergent lines of development with lightweight movable pointers

5 Remotes, Syncing & Resolving Conflicts
Connecting to remote repositories, sharing commits, and resolving merge conflicts

6 Git Basics Cheat Sheet & Interview Guide
12 key concepts, 10 must-know interview questions, and the ultimate Git formula

Core Learning Pathway



How to use these notes

- Page 3 sets the ***3-stage architecture*** — master Working Tree, Index & Repo before memorising commands.
- Page 4 covers ***branching, pointers & merging*** — how `switch` & `restore` cleanly replace legacy `checkout`.
- Page 5 covers ***syncing with remotes*** — the mechanics of `fetch`, `pull`, `push`, and resolving merge conflicts.
- Page 6 is your ***cheat sheet & interview guide*** — 12 mini-guides, 10 questions, and the ultimate Git formula.
- Do the sample commands yourself in a sandbox directory to build muscle memory.

Mindset

- Git does not store diffs; it stores full ***snapshots*** of your project on every commit.
- Commits form a ***Directed Acyclic Graph (DAG)*** where each node points back to parent commits.
- Nearly every operation is ***local-first***; network latency only matters during `fetch`, `pull`, and `push`.
- Git rarely deletes data. If you lose work, `git reflog` can usually rescue orphaned commits.

Tools you will need

- ***Git CLI:*** Version 2.23+ recommended to use `switch` & `restore` syntax.
- ***Terminal Shell:*** zsh or bash with Git branch auto-completion enabled.
- ***Hosting Platform:*** GitHub, GitLab, or Bitbucket for remote collaboration.
- ***Visual DAG Viewers:*** Built-in `git log --graph --oneline` or VS Code Git Graph extension.
- ***Productivity Aliases:*** Setup `git config --global alias.st status` and `alias.co checkout`.

Modern Git Standards

- ***Default Branch:*** Now `main` instead of `master` (`git config --global init.defaultBranch main`).
- ***Replaced checkout:*** Modern Git uses `git switch` (branch navigation) and `git restore` (undoing changes).
- ***Atomic Commits:*** Make small, focused commits with imperative messages (e.g. `feat: add user auth`).
- ***Never Rebase Public Branches:*** Only rebase local unpublished branches to avoid rewriting shared history.

Real-world use cases
Self-study, interview prep and quick revision.

Interview tip
Learn the vocabulary first; definitions come up in every interview.

Pro tip
Keep a running notes file of anything you look up.

The Three Trees: Work, Stage & Commit

GIT
VCS Basics

Understanding how Git moves changes through working directory, index, and repository

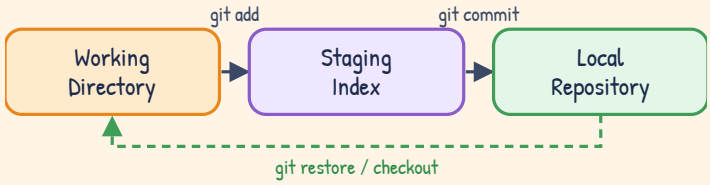
1 What is Git?

- ***Distributed VCS:*** Every developer clone holds the entire project history and complete object graph locally on disk.
- ***Snapshot Model:*** Unlike legacy VCS tracking diff deltas, Git captures immutable ***snapshots*** of the directory tree.
- ***Cryptographic Integrity:*** Every object is content-addressed by a 40-character SHA hash, preventing silent corruption.

2 The Three Areas

- ***Working Directory:*** Local sandbox files you edit directly in your code editor.
- ***Staging Area (Index):*** Binary cache (`.git/index`) preparing the exact snapshot for commit.
- ***Repository (.git):*** Permanent object database storing verified snapshots & history graph.

3 The Lifecycle Flow



4 Behind the Scenes (.git object store)

- ***Blob:*** Compressed raw file content (stores zero filenames or permissions).
- ***Tree:*** Represents a directory; maps filenames & modes to child blob/tree SHAs.
- ***Commit:*** Points to root tree SHA, parent commit(s), author, timestamp, and message.
- ***Annotated Tag:*** Permanent named reference pointing directly to a specific commit.

5 Core Staging Commands

Command	Action	Target Area
git init	Create new local repository	.git folder
git status	Check modified vs staged files	Work vs Index
git add <file>	Stage specific file for commit	Index buffer
git add -p	Interactively stage hunks (patches)	Index buffer
git diff	View unstaged changes vs Index	Workdir diff
git commit -m	Record snapshot into history	Repository

6 The File States & Lifecycle

State	Meaning	Next Action
Untracked	New file not yet registered in Git	git add <file>
Unmodified	File matches last commit snapshot	Edit file
Modified	Edited in workdir, not yet staged	git add / restore
Staged	Indexed buffer ready for commit	git commit
Ignored	Matches rule in .gitignore file	Skipped by Git

7 First Commit Walkthrough

Terminal workflow

```
# 1. Initialize repository and create file
git init my-project && cd my-project
echo "# Git Study Notes" > README.md
# 2. Stage file to Index and record snapshot
git add README.md
git commit -m "feat: initial commit"
```

Sample terminal output

```
[main (root-commit) 4a9b1c2] feat: initial commit
 1 file changed, 1 insertion(+)
 create mode 100644 README.md
```

8 Common Mistakes & Fixes

- Editing files and forgetting to stage. Fix: Check `git status` before running commit.
- Committing secrets, .env, or node_modules. Fix: Add patterns to `.gitignore` before initial add.
- Vague commit messages like 'fixed stuff'. Fix: Write clear imperative summaries like `feat: add user login`.
- Accidentally staged the wrong file. Fix: Run `git restore --staged <file>` to unstage without losing edits.

9 Remember

Edit in Working Tree, Buffer in Staging Index, Save permanently in Repository Snapshot.

Real-world use cases

Atomic commits allow easy rollbacks, clear code reviews, and clean bisecting when bugs arise.

Interview tip

Q: What is the Staging Area? A: An intermediate buffer allowing selective, atomic commits.

Pro tip

Use `git add -p` (patch mode) to review and stage specific chunks of code interactively.

Branching, Merging & Moving HEAD

GIT
VCS Basics

How Git tracks divergent lines of development with lightweight movable pointers

1 What is a Branch & HEAD?

- **Pointer Model:** A branch is simply a lightweight 41-byte text file storing a 40-hex commit hash (`.git/refs/heads/<name>`).
- **Zero Overhead:** Creating or switching branches is instant; zero files are duplicated.
- **HEAD Cursor:** References whichever branch or commit is currently checked out (`.git/HEAD`).
- **Auto-Advance:** When you commit, Git automatically advances the current branch pointer.

2 Branch Topology & Pointer Flow



3 Modern Navigation (Git 2.23+)

Task	Modern Command	Legacy Form
Switch branch	git switch <name>	git checkout <name>
Create & switch	git switch -c <name>	git checkout -b <name>
Discard work edits	git restore <file>	git checkout -- <file>
Unstage a file	git restore --staged <f>	git reset HEAD <file>
List all branches	git branch -av	git branch -a

4 Merge Strategies Compared

Strategy	Behavior	History Impact
Fast-Forward	Moves branch label forward	Zero extra commits
3-Way Merge	Combines 2 divergent heads	Preserves merge commit
Rebase	Replays commits on top of base	Linear clean graph
Squash Merge	Condenses branch into 1 commit	Single clean commit

5 Feature Branch Workflow

Branch & Merge Workflow

```
# 1. Create and isolate feature branch
git switch -c feature/auth-token
# ... edit files and commit ...
git commit -am "feat: add login token validation"
# 2. Switch back and integrate into main
git switch main
git merge --no-ff feature/auth-token
```

3-way merge output

```
Merge made by the 'ort' strategy.
 auth.py | 24 ++++++++++++++++++++---
 1 file changed, 21 insertions(+), 3 deletions(-)
```

6 Saving Work with Stash

- `git stash -u` : Shelves dirty changes (including untracked files).
- `git stash pop` : Re-applies the most recent stash and deletes it.
- `git stash list` : Inspects all saved stash entries across branches.
- `git stash drop` : Permanently discards a specific saved stash state.

7 Common Traps & Fixes

- Detached HEAD state. Fix: Run `git switch main`.
- Rebasing shared branches. Fix: Never rebase commits pushed to remote.
- Branch switch blocked. Fix: Run `git stash`, switch, then `git stash pop`.
- Wrong branch commit. Fix: `git cherry-pick <hash>` on target, then reset.

8 Remember

Branches are 41-byte pointers; HEAD is where you stand. Switch cleanly with git switch.

Real-world use cases

Feature branches isolate experimental code without touching stable production main.

Interview tip

Q: Git Merge vs Rebase? A: Merge preserves historical context; rebase rewrites linear history.

Pro tip

Run `git log --oneline --graph --all` to view your branch topology in the terminal.

Remotes, Syncing & Resolving Conflicts

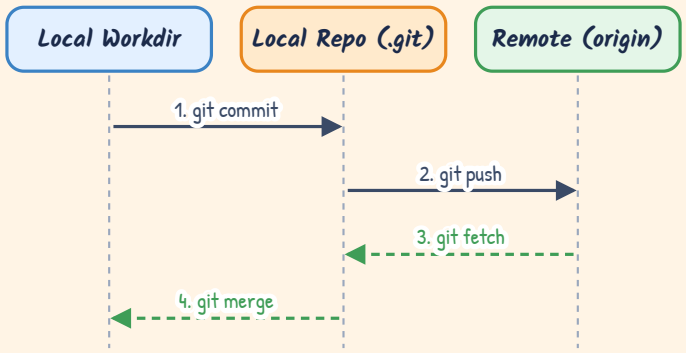
GIT
VCS Basics

Connecting to remote repositories, sharing commits, and resolving merge conflicts

1 What is a Remote?

- ***Remote Server:*** A repository hosted on a network or cloud platform (GitHub, GitLab, Bitbucket).
- ***Origin Convention:*** Git automatically names the default upstream remote `origin` (`.git/config`).
- ***Tracking Branches:*** References like `origin/main` are read-only local caches of remote state.
- ***Offline-First:*** All commit and diff operations occur offline; network sync is explicit.

2 Local vs Remote Sync Flow



3 Syncing Commands

Command	What it does	Affects Workdir?
git fetch	Downloads remote commits & refs	No (100% safe)
git merge origin/main	Merges fetched branch into local	Yes
git pull	fetch + merge combined in 1 step	Yes
git pull --rebase	fetch + rebase local atop remote	Yes (clean graph)
git push -u origin main	Uploads commits and sets upstream	No

4 When Conflicts Happen

- ***Divergent Edits:*** Occurs when two branches edit the exact same lines, or one deletes an edited file.
- ***Automatic Pause:*** Git halts the merge and injects markers into affected files in the workdir.
- ***Conflict Markers:*** `<<<<<<< HEAD` (your code), `=======` (divider), and `>>>>>>>` (incoming code).
- ***Resolution Flow:*** Choose desired lines, remove markers, stage with `git add`, and finalize with `commit`.

5 Conflict Anatomy & Resolution

Conflict markers & resolution

```
# 1. Inside conflicted settings.py:
<<<<<<< HEAD
DATABASE_PORT = 5432  # Production port
=======
DATABASE_PORT = 5433  # Staging update
>>>>>>> feature/port-update

# 2. Pick desired line, delete markers, then:
git add settings.py
git commit -m "fix: resolve database port conflict"
```

Resolution commit output

```
[main 9a2b1c4] fix: resolve database port conflict
 1 file changed, 1 insertion(+), 1 deletion(-)
```

6 Remote Best Practices

- ***Set Upstream Tracking:*** Use `git push -u origin <branch>` once.
- ***Rebase Before Push:*** `git pull --rebase` avoids messy merge bubbles.
- ***Small Focused PRs:*** Keep PRs under 300 lines for fast review.
- ***Protected Branches:*** Mandate peer reviews and CI checks on `main`.

7 Remote Sync Mistakes & Fixes

- Push rejected (non-fast-forward). Fix: Run `git pull --rebase`, then push.
- Raw `git push --force` on main. Fix: Use `--force-with-lease`.
- Panic on conflict markers. Fix: Run `git merge --abort`.
- Pushed to wrong remote branch. Fix: Set `push.default simple`.

8 Remember

Fetch is safe inspection; Pull is fetch plus merge. Resolve markers, stage, and commit to settle conflicts.

Real-world use cases

Continuous Integration (CI) systems trigger builds and test suites on git push events.

Interview tip

Q: git fetch vs git pull? A: Fetch downloads objects without modifying working files; pull merges them.

Pro tip

Configure `git config --global pull.rebase true` to keep pull histories clean and linear.

Git Basics Cheat Sheet & Interview Guide

GIT
VCS Basics

12 key concepts, 10 must-know interview questions, and the ultimate Git formula

1 Snapshot Model

- Stores full trees, never diff deltas.
- Unchanged files are hard-linked.
- Commits are immutable snapshots.
- Keyed by 40-character SHA hash.

2 Three States

- Modified: files edited in workdir.
- Staged: snapshot buffered in Index.
- Committed: saved in local repository.
- Move states via `add` and `commit`.

3 Branch Pointers

- 41-byte pointer to commit SHA.
- Microsecond creation & zero bloat.
- `HEAD` references active branch.
- `git switch -c <name>` to branch.

4 switch & restore

- `git switch <branch>`: jump branch.
- `git restore <file>`: discard edits.
- `git restore --staged`: unstage.
- Modern replacement for checkout.

5 Merging Types

- Fast-Forward: moves branch label.
- 3-Way: combines ancestor + heads.
- Squash: collapses branch to 1 commit.
- Abort with `git merge --abort`.

6 Rebase vs Merge

- Merge preserves historic timeline.
- Rebase rewrites linear graph.
- Interactive: `git rebase -i HEAD~3`.
- Rule: Never rebase shared branches.

7 Remote Sync

- `origin`: default remote server.
- `git fetch`: safe remote download.
- `git pull`: fetch + merge in 1 step.
- `git push -u origin <branch>`: track.

8 Stashing Work

- `git stash`: shelves dirty workdir.
- `git stash -u`: include untracked.
- `git stash pop`: restore & drop.
- `git stash list`: view all entries.

9 Undoing Mistakes

- `git commit --amend`: edit last commit.
- `git revert <hash>`: safe public undo.
- `git reset --soft`: keep edits in index.
- `git reset --hard`: destructive reset.

10 Conflict Markers

- `<<<<<<< HEAD`: your local code.
- `=======`: conflict separator.
- `>>>>>>>`: incoming branch code.
- Edit, remove markers, stage & commit.

11 The Reflog

- Log of every `HEAD` move on disk.
- Safety net: rescues lost commits.
- Recovers deleted branch commits.
- Recover: `git switch -c rescue <h>`.

12 Git Aliases

- Custom shortcuts in `~/.gitconfig`.
- `alias.st status` | `alias.sw switch`.
- `alias.lg "log --oneline --graph"`.
- Boosts daily developer velocity.

Top 10 Interview Questions

1. ***What is the difference between Git and SVN?*** Git is distributed with full local snapshots; SVN is centralised and diff-based.
2. ***What happens internally when you commit?*** Git writes blob objects for files, a tree object for directories, and a commit object with metadata.
3. ***What is the staging area (index) and why does it exist?*** It provides an intermediate buffer allowing developers to craft atomic, granular commits.
4. ***Explain git fetch vs git pull.*** Fetch updates remote-tracking references without touching working files; pull fetches and immediately merges.
5. ***How does Fast-Forward merge differ from 3-Way merge?*** Fast-forward moves the branch label forward linearly; 3-way creates a merge commit from a common ancestor.
6. ***What is git rebase and when is it dangerous?*** It replays commits on top of another base to linearize history. Dangerous if run on shared public branches.
7. ***What is a detached HEAD state?*** HEAD points directly to a commit hash rather than a named branch; new commits risk being orphaned.
8. ***How do you resolve a merge conflict?*** Open the conflicted file, manually choose desired lines, delete `<<<<<<<` and `>>>>>>>` markers, stage, and commit.
9. ***Difference between git reset, git revert, and git restore?*** Reset rewrites branch pointer; revert creates an inverse commit; restore affects working tree / index.
10. ***How can you recover a commit lost to git reset --hard?*** Run `git reflog` to identify the orphaned commit hash, then `git switch -c recovery <hash>`.

The Ultimate Git Formula

Snapshot -> Stage -> Commit -> Branch -> Fetch -> Merge (Recover with Reflog)

Real-world use cases

Daily reference sheet for code reviews, onboarding engineers, and resolving git blocks.

Interview tip

Mastering the DAG mental model answers 90% of all Git technical interview questions.

Pro tip

When stuck, run git status and git reflog before running destructive commands.